

SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

Getthawha Omnichannel Spa & Wellness Booking Platform

Project Title	Getthawha Omnichannel Spa & Wellness Platform
Document Type	Software Requirements Specification (SRS)
Standards Compliance	ISO/IEC/IEEE 29148:2018 / IEEE Std 830-1998
Quality Model Alignment	ISO/IEC 25010 Software Quality Standards
Document Version	Version 2.0 (Comprehensive Baseline)
Target Organization	Getthawha Thai Massage & Wellness Group, Chiang Mai
Technical Authority	Lead Systems Analyst & Solutions Architect
Effective Date	September 2026
Classification	Commercial Confidential / Engineering Specification

Document Control & Approvals

This document constitutes a baseline engineering deliverable for the Getthawha Platform. Formal review, authorization, and historical modifications are recorded below:

Formal Review & Acceptance Sign-Off

Role / Representation	Name	Title / Organization	Status	Sign Date
Product Owner	Mawin (Project Owner)	Executive Stakeholder	Approved	September 2026
Systems Analysis	Lead Systems Analyst	Solutions Architecture	Approved	September 2026
Engineering	Lead Software Engineer	Fullstack Development Team	Approved	September 2026
Verification	QA Lead	Software Quality Assurance	Approved	September 2026

Document Revision History

Version	Release Date	Author / Role	Summary of Technical Changes
1.0	August 10, 2026	Systems Analyst	Initial requirements baseline for online booking and admin management
1.5	August 30, 2026	QA / Product Specialist	Integrated Google OAuth 2.0 PKCE and LINE LIFF specifications
2.0	September 14, 2026	Software Engineering Team	Comprehensive expansion: 15 Functional Requirements, Regex Validation Matrix, and RTM

Confidentiality & Proprietary Baseline

The information contained in this document is confidential and proprietary to Getthawha Thai Massage & Wellness Group. It is submitted with the understanding that it will be held in strict confidence and will not be disclosed, duplicated, or used, in whole or in part, for any purpose other than evaluating and operating the Getthawha Omnichannel Platform without prior written consent from the executive board.

Table of Contents

This specification is organized into the following formal sections:

1.0	Introduction & Document Conventions
2.0	Overall System Description & Persona Matrix
3.0	Specific Functional Requirements (FR-01 through FR-15)
4.0	Form Input Validation & Business Rule Matrix
5.0	External Interface Requirements (API, GUI, Network)
6.0	Non-Functional Requirements (ISO/IEC 25010 Quality Model)
7.0	Requirements Traceability Matrix (RTM)

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) establishes the complete, verifiable, and unambiguous requirements for the **Getthawha Omnichannel Spa & Wellness Booking Platform**. It outlines both the functional capabilities and non-functional quality constraints necessary for implementation, testing, and operational acceptance.

1.2 Document Conventions

- **Functional Requirements (`FR-XXX`)**: Specify verifiable system behaviors and interfaces.
- **Non-Functional Requirements (`NFR-XXX`)**: Specify quality attributes governed by the ISO/IEC 25010 model.
- **MoSCoW Priority Ratings**:
 - **Must Have (M)**: Mandatory core requirements for production release.
 - **Should Have (S)**: High-priority features with manual operational workarounds.
 - **Could Have (C)**: Value-add enhancements if delivery schedules allow.
 - **Won't Have (W)**: Deferred to future development phases.
- **Acceptance Criteria**: Formatted using standard **Gherkin syntax** (**Scenario**, **Given**, **When**, **Then**).

1.3 Intended Audience

- **Frontend & Backend Engineers**: Technical implementation specifications and interface contracts.
- **Quality Assurance Engineers**: Test plan formulation, automated test scripting, and UAT validation.
- **Store Operations & Management**: Verification of business workflows and front-desk operational rules.

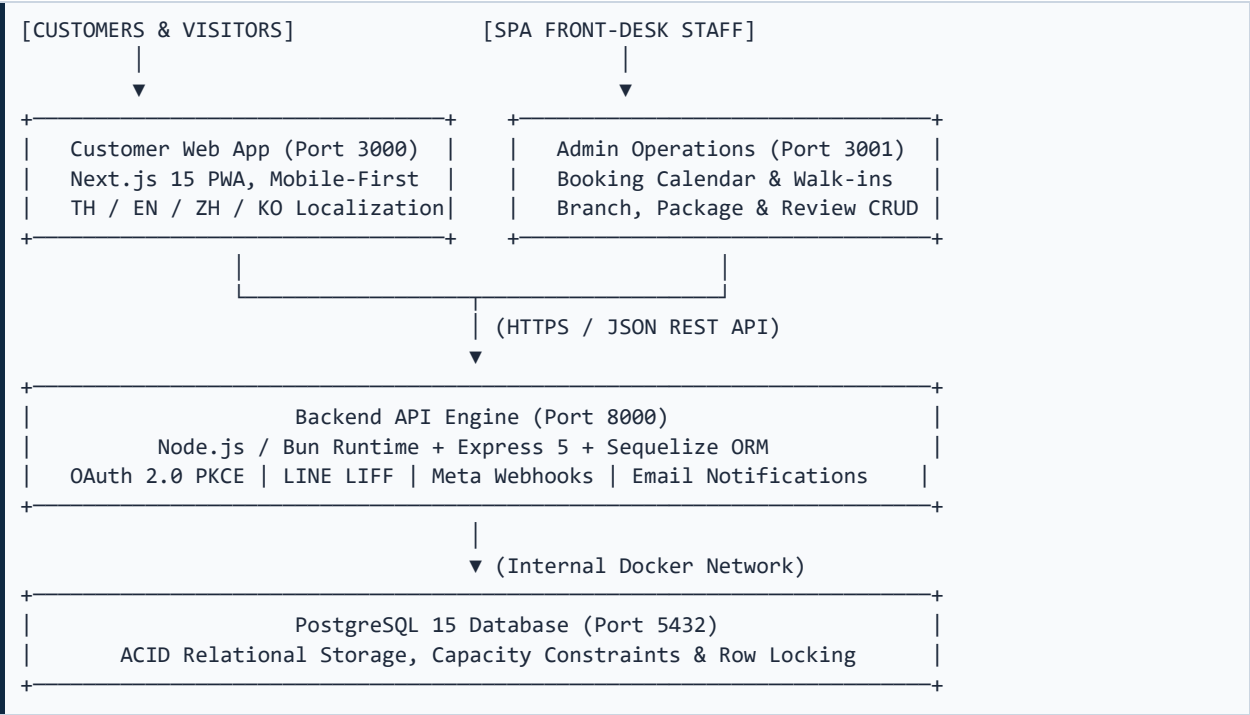
1.4 Terminology & Glossary

- **ACID**: Atomicity, Consistency, Isolation, Durability.
- **CJK**: Chinese, Japanese, and Korean typography and script rendering.
- **CORS**: Cross-Origin Resource Sharing security policy.
- **Haversine Formula**: Mathematical formulation calculating great-circle distances between two geographic coordinates on a spherical surface.
- **JWT**: JSON Web Token (RFC 7519).
- **LIFF**: LINE Front-end Framework (web view platform embedded within the LINE mobile app).
- **PKCE**: Proof Key for Code Exchange (RFC 7636 security extension for OAuth 2.0).
- **RTM**: Requirements Traceability Matrix.
- **SSO**: Single Sign-On.

2. Overall Description

2.1 Product Perspective & Context

The platform functions as an integrated software suite connecting customer touchpoints to a centralized operational backend:



2.2 User Roles & Persona Matrix

Persona ID	User Role	Description & Typical Actions	Access Clearance
USR-01	Unauthenticated Guest	Browses treatment catalogs, locates nearby branches via GPS, views pricing and promo cards, reads verified reviews, and initiates login.	Public / Anonymous
USR-02	Authenticated Customer	Logged in via Google OAuth or LINE Login. Selects services, applies voucher discounts, confirms reservations, views history, and cancels bookings.	User Session JWT
USR-03	Branch Front-Desk Staff	Receptionist managing daily schedules, checking in arriving customers, and recording manual reservations for walk-in or telephone guests.	Staff Session JWT
USR-04	System Administrator	Business owner or head office manager managing branches, packages, promo codes, staff credentials, and moderating customer reviews.	Admin Session JWT
USR-05	Automated Webhooks	Meta Graph API crawler delivering messaging events and verifying webhook subscriptions.	Token Verified Handshake

2.3 Operating Environment & Technical Constraints

- **Client Devices:** Optimized for mobile viewports (iOS Safari, Android Chrome, LINE in-app webview, WeChat browser) down to 360px screen width, as well as desktop web browsers.
- **Server Environment:** Docker Engine 24.0+ running on Linux (Ubuntu 22.04 LTS / Debian 12).
- **System Timezone:** Strictly synchronized to **`Asia/Bangkok` (UTC+7)** across all database queries, calendar slot renders, and logging timestamps.
- **Port Bindings:** Client: Port 3000, Admin: Port 3001, API: Port 8000, Database: Port 5432.

3. Specific Functional Requirements (FR)

Module 1: Authentication, Identity & Access Control

`FR-AUTH-01`: LINE Login & LIFF In-App Authentication

- **Priority:** Must Have (M)
- **Actor:** USR-02 (Customer)
- **Description:** Customers opening the application via LINE LIFF or selecting LINE Login authenticate via LINE OAuth 2.1. The backend verifies the `id_token` with LINE's OAuth verification endpoint (<https://api.line.me/oauth2/v2.1/verify>), retrieves the user's `sub` identifier, display name, and avatar picture, creates or updates the record in the `users` table, and sets an encrypted JWT in an `HttpOnly` cookie named `info`.
- **Pre-conditions:** Customer has access to LINE App or web browser.
- **Main Success Flow:**
 1. Customer taps "Sign in with LINE" or opens the booking LIFF link.
 2. LIFF SDK retrieves the customer's ID token.
 3. Client dispatches `POST /line/authorization` with `idToken`.
 4. Backend verifies signature and claims with LINE servers.
 5. System provisions user record in PostgreSQL.
 6. Backend generates signed user JWT and writes `info` cookie.
 7. Client redirects customer to the active reservation flow.
- **Acceptance Criteria (Gherkin):**

```
Scenario: Customer authenticates seamlessly via LINE LIFF
  Given an unauthenticated customer opens the booking portal inside LINE
  When the LIFF SDK retrieves a valid ID token
  And dispatches it to "/line/authorization"
  Then the database stores the user ID and display name
  And an HttpOnly cookie "info" containing a valid JWT is issued
  And the user is redirected to their active reservation flow
```

`FR-AUTH-02`: Google OAuth 2.0 Single Sign-On with PKCE

- **Priority:** Must Have (M)
- **Actor:** USR-02 (Customer)
- **Description:** The system supports Google OAuth 2.0 with Proof Key for Code Exchange (PKCE, RFC 7636).
- **Main Success Flow:**
 1. Browser calls `GET /google/authentication?next=/booking?packageId=123`.
 2. Backend generates cryptographically secure `state` (32 chars) and `code_verifier` (64 bytes).
 3. Backend computes `code_challenge = Base64URL(SHA256(code_verifier))`.
 4. Backend sets temporary cookies: `googleState`, `googleVerifier`, and `loginReturn`.
 5. Browser redirects to Google's consent dialog.

6. Customer authorizes access; Google redirects to `GET /google/authorization?code=XYZ&state=...`
7. Backend verifies `state` parameter against cookie, exchanges `code` and `code_verifier` with Google.
8. Google returns verified OpenID token; backend provisions user record in database.
9. Backend issues user JWT, sets `info` cookie, and redirects user to `loginReturn` path.

- **Acceptance Criteria (Gherkin):**

```
Scenario: Customer authenticates via Google OAuth with PKCE
  Given an unauthenticated customer selects "Sign in with Google"
  When the PKCE state and SHA-256 code challenge are verified
  And Google authorization succeeds
  Then a new user record is created in PostgreSQL with Google sub identifier
  And the customer is redirected back to their selected package with active session
```

`FR-AUTH-03`: Staff & Administrative Authentication

- **Priority:** Must Have (M)
- **Actor:** USR-03 (Staff), USR-04 (Admin)
- **Description:** Staff and administrators authenticate via `POST /admin/auth/login` using username and password. The system verifies passwords against Bcrypt hashes stored in `userStaffs` (cost factor 10) and returns a signed Admin JWT.
- **Acceptance Criteria (Gherkin):**

```
Scenario: Administrator logs into Admin Portal
  Given a valid administrator with username "admin"
  When submitting the correct password
  Then the server returns an admin JWT
  And the user is granted access to the master calendar and operations
```

Module 2: Booking Engine & Service Catalog

`FR-BOOK-01`: Interactive Booking Wizard

- **Priority:** Must Have (M)
- **Actor:** USR-02 (Customer)
- **Description:** The client frontend provides an intuitive multi-step booking wizard:
 1. **Branch Selection:** Choice of 5 Chiang Mai branches with photos, address, and distance indicators.
 2. **Package & Duration Selection:** Service package with duration toggle (60m, 90m, 120m).
 3. **Date & Timeslot:** Appointment date and available time slot between 09:30 and 20:00.
 4. **Customer Information:** Full Name, Mobile Phone, and Email address.
 5. **Voucher Code:** Optional promo discount code.
- **Acceptance Criteria (Gherkin):**

```
Scenario: Multi-step booking wizard completion
```

```

Given a customer has chosen branch "Getthawa Spa (Main Branch)"
When selecting date "2026-10-01" and time "14:00"
And submitting customer details "John Doe", "0812345678"
Then the system proceeds to summary review with final price

```

`FR-BOOK-02`: Dynamic Duration Selector & Pricing Engine

- **Priority:** Must Have (M)
- **Actor:** USR-02 (Customer)
- **Description:** Treatment durations are configurable per service (60, 90, 120 minutes). When the customer switches duration, the frontend dynamically recalculates the price, and the backend verifies the duration-price mapping server-side in `booking.validation.js` before inserting the record into the database.
- **Acceptance Criteria (Gherkin):**

```

Scenario: Duration change dynamically adjusts price
  Given a package "Traditional Thai Massage" with base 60-minute price 600 THB
  When customer switches duration to "120" minutes
  Then the displayed and verified price updates to 1200 THB

```

`FR-BOOK-03`: Voucher & Promo Code Verification

- **Priority:** Should Have (S)
- **Actor:** USR-02 (Customer)
- **Description:** Customers can enter promotional voucher codes. The system checks that the code exists in `vouchers` and that `isExpired` is false. The discount is deducted from the treatment price:

$\text{Final Net Price} = \max(0, \text{Package Price} - \text{Voucher Discount})$

- **Acceptance Criteria (Gherkin):**

```

Scenario: Valid voucher reduces total price
  Given a booking with subtotal 1500 THB
  When applying a valid voucher with 300 THB discount
  Then the net payable total displays as 1200 THB

```

`FR-BOOK-04`: Transactional Email Notifications

- **Priority:** Should Have (S)
- **Actor:** System
- **Description:** When an appointment is created with status `pending` or `confirmed`, the system triggers `sendNotifications.js`, dispatching a formatted HTML confirmation email containing appointment UUID, branch address, Google Maps link, and cancellation terms.
- **Acceptance Criteria (Gherkin):**

```

Scenario: Automatic email confirmation dispatched
  Given a completed booking with ID "uuid-booking-123"
  When the database transaction commits
  Then an HTML confirmation email is dispatched to the customer's registered email address

```

`FR-BOOK-05`: Customer Profile & Self-Service Cancellation

- **Priority:** Must Have (M)
- **Actor:** USR-02 (Customer)
- **Description:** Customers can review their upcoming and past bookings under `/profile`. Upcoming bookings may be cancelled via `PATCH /booking/:id/cancel`, changing status to `cancelled` and releasing the reserved room and therapist capacity.
- **Acceptance Criteria (Gherkin):**

```

Scenario: Customer cancels pending reservation
  Given a customer with an active booking in status "pending"
  When the customer confirms cancellation in their profile
  Then the database status updates to "cancelled"
  And the slot becomes available for re-booking

```

Module 3: Geolocation & Branch Experience**`FR-BRAN-01`: Geolocation Nearest Branch Detection**

- **Priority:** Should Have (S)
- **Actor:** USR-01 (Guest), USR-02 (Customer)
- **Description:** The system requests browser GPS coordinates via HTML5 Geolocation API. Upon consent, the client calculates the distance to each of the 5 branches using the Haversine formula:

$$\text{Distance (km)} = 2 \times R \times \arcsin\left(\sqrt{\sin^2(\Delta\text{lat} / 2) + \cos(\text{lat}_1) \times \cos(\text{lat}_2) \times \sin^2(\Delta\text{lon} / 2)}\right)$$

where $R = 6,371$ km. The closest branch is highlighted with a "Nearest Branch" badge and distance in kilometers.

- **Acceptance Criteria (Gherkin):**

```

Scenario: Geolocation detects nearest branch in Nimman
  Given a user located at coordinates (18.7961, 98.9691) in Nimman
  When location permission is granted
  Then the "Getthawha Nimman Branch" is highlighted as nearest with calculated distance

```

Module 4: Social Commerce & Conversational Bot**`FR-FBOT-01`: Meta Facebook Messenger Bot Webhook**

- **Priority:** Should Have (S)
- **Actor:** USR-05 (Meta Webhook), Facebook Messenger User
- **Description:** The system exposes `/facebook/webhook`:

1. **Verification (GET):** Validates `hub.verify_token` against `FB_VERIFY_TOKEN` and echoes `hub.challenge`.
2. **Messaging Event (POST):** Receives incoming customer messages and replies via Meta Graph API v21 with Quick Reply options (Treatment Menu, Branch Locations, and Online Booking link).

- **Acceptance Criteria (Gherkin):**

```
Scenario: Facebook Messenger delivers Quick Replies
  Given a Facebook user asks "How much is Thai massage?"
  When Meta delivers the messaging webhook POST to "/facebook/webhook"
  Then the backend dispatches a reply containing menu prices and a "Book Now" quick action
```

Module 5: Administrative Operations & Store Management

`FR-ADMN-01`: Master Booking Calendar & Schedule Grid

- **Priority:** Must Have (M)
- **Actor:** USR-03 (Staff), USR-04 (Admin)
- **Description:** Displays monthly booking density (`GET /admin/calendar/:year/:month`) and chronological daily appointment rosters (`GET /admin/calendar/date/:d/:m/:y`), showing customer names, selected treatments, durations, and statuses.
- **Acceptance Criteria (Gherkin):**

```
Scenario: Viewing daily timeslots in Admin Calendar
  Given an administrator opens the calendar for date "2026-10-05"
  When the daily roster loads
  Then all confirmed and pending appointments for all 5 branches are displayed in chronological order
```

`FR-ADMN-02`: Walk-in & Telephone Manual Booking

- **Priority:** Must Have (M)
- **Actor:** USR-03 (Staff)
- **Description:** Front-desk staff record offline walk-in and phone reservations via `/manualBooking`, selecting an existing user or guest profile, branch, treatment, and timeslot without requiring online payment or social login.
- **Acceptance Criteria (Gherkin):**

```
Scenario: Staff records walk-in customer booking
  Given a walk-in guest at Chiang Mai Main Branch
  When the receptionist enters customer name "Alice Smith" and selects "Foot Massage"
  Then a new confirmed booking is created immediately in PostgreSQL
```

`FR-ADMN-03`: Master Data Management (CRUD)

- **Priority:** Must Have (M)
- **Actor:** USR-04 (Admin)

- **Description:** Full management interfaces for:
- **Branches:** Name, address, phone, Google Maps embed URL, active status.
- **Packages:** Title, description, price, duration, image upload (**/upload**), and promotional classification.
- **Vouchers:** Code creation, discount amount, and expiry toggle.
- **Staff Accounts:** Provisioning staff accounts with Bcrypt password hashing.
- **Acceptance Criteria (Gherkin):**

```
Scenario: Administrator creates a new seasonal promotion package
  Given an authenticated administrator on "/packageManagement"
  When submitting a new package titled "Lanna Herbal Warm Compress" with price 1200 THB
  Then the new package is saved to PostgreSQL and immediately visible on the customer web catalog
```

`FR-ADMN-04`: Customer Review Moderation

- **Priority:** Should Have (S)
- **Actor:** USR-04 (Admin)
- **Description:** Customer reviews submitted online remain pending (**isApproved: false**) until approved by an administrator in the Review Management dashboard via **PATCH /admin/review/:id/approve**.
- **Acceptance Criteria (Gherkin):**

```
Scenario: Moderating customer reviews
  Given a new customer review with 5 stars and status unapproved
  When the administrator clicks "Approve"
  Then the database sets isApproved to true and the review appears publicly
```

Module 6: Quadrilingual Internationalization (i18n)

`FR-I18N-01`: Multi-Language Interface & Dynamic Typography

- **Priority:** Must Have (M)
- **Actor:** USR-01 (Guest), USR-02 (Customer)
- **Description:** The system supports immediate client-side locale toggling across four languages: Thai (**th**), English (**en**), Simplified Chinese (**zh**), and Korean (**ko**). **LocaleFont.tsx** dynamically applies matching typography:
- Thai: Kanit / Prompt
- English: Inter / Geist Sans
- Chinese: Noto Sans SC
- Korean: Noto Sans KR
- **Acceptance Criteria (Gherkin):**

```
Scenario: Switching language to Simplified Chinese
  Given a customer viewing the booking page in English
  When selecting "中文 (Chinese)" from the language menu
```

Then all headers, package names, duration selectors, and buttons switch to Simplified Chinese
And the typography adjusts to Noto Sans SC

4. Comprehensive Input Validation Matrix

Field Name	Target Entity	Data Type	Mandatory	Validation Rule & Format	Error Message on Validation Failure
username	userStaffs	String	Yes	3 to 30 alphanumeric characters (<code>^[a-zA-Z0-9_-]{3,30}\$</code>)	"Username must be 3-30 alphanumeric characters."
email	userStaffs	String	Yes	Standard email format with valid domain name	"Please provide a valid email address."
password	userStaffs	String	Yes	Minimum 8 characters, containing at least 1 letter and 1 number	"Password must be at least 8 characters with letters & numbers."
date	bookings	Timestamp	Yes	ISO 8601 string, future date, timeslot between 09:30 and 20:00	"Appointment date must be a valid future timeslot between 09:30 and 20:00."
totalPrice	bookings	Decimal	Yes	Positive decimal number (≥ 0.00)	"Total price cannot be negative."
duration	packages	Integer	Yes	Value must be 60, 90, 120, or 180 minutes	"Duration must be 60, 90, 120, or 180 minutes."
code	vouchers	String	Yes	3 to 20 uppercase alphanumeric characters (<code>^[A-Z0-9_-]{3,20}\$</code>)	"Voucher code must be 3-20 uppercase characters."
discount	vouchers	Decimal	Yes	Decimal number greater than 0.00	"Discount must be greater than zero."
rating	reviews	Integer	Yes	Integer between 1 and 5	"Rating must be between 1 and 5 stars."
phone	branches	String	Yes	9 to 15 digit telephone format (<code>^[0-9+ -]{9,15}\$</code>)	"Please enter a valid phone number."

5. Non-Functional Requirements (ISO/IEC 25010 Quality Model)

5.1 Security Requirements (NFR-SEC)

- **NFR-SEC-01 (Cookie Security):** All user and admin session JWTs are stored exclusively in `HttpOnly`, `SameSite=Lax` cookies. On production (`NODE_ENV=production`), the `Secure=true` flag is mandatory.
- **NFR-SEC-02 (Password Hashing):** All staff passwords stored in `userStaffs` are hashed with Bcrypt (work factor 10). Plaintext passwords must never be stored or logged.
- **NFR-SEC-03 (SQL Injection Defense):** All database operations execute through Sequelize ORM parameterized queries to eliminate SQL injection vulnerabilities.
- **NFR-SEC-04 (File Upload Restrictions):** Media uploaded via `/upload` is restricted to authenticated administrators, verified against MIME types (`image/jpeg`, `image/png`, `image/webp`), capped at 5 MB, and assigned randomized UUID filenames.

5.2 Performance Efficiency Requirements (NFR-PERF)

- **NFR-PERF-01 (API Response Times):** 95% of catalog and branch read requests (`GET /package`, `GET /branch`) respond in ≤ 120 ms under a concurrent load of 50 users.
- **NFR-PERF-02 (Core Web Vitals):** The customer booking web application targets:
 - Largest Contentful Paint (LCP) ≤ 2.2 seconds.
 - Cumulative Layout Shift (CLS) ≤ 0.05 .
 - First Input Delay (FID) ≤ 80 ms.

5.3 Reliability & Availability (NFR-REL)

- **NFR-REL-01 (System Availability):** The platform maintains $\geq 99.8\%$ uptime during business operating hours (08:00 to 22:00 Asia/Bangkok time).
- **NFR-REL-02 (Container Self-Healing):** All Docker service containers (`postgres`, `backend`, `client-frontend`, `admin-frontend`) specify `restart: always` with active PostgreSQL healthchecks.

5.4 Usability & Accessibility (NFR-USAB)

- **NFR-USAB-01 (Mobile Responsiveness):** All booking wizard steps are fully operable on smartphone screens down to 360px width without horizontal overflow.
- **NFR-USAB-02 (Reservation Efficiency):** An authenticated customer can complete an appointment reservation in 4 screen interactions or fewer from the service page.

5.5 Maintainability & Portability (NFR-MAINT)

- **NFR-MAINT-01 (Container Portability):** The multi-container stack runs locally or on any cloud VPS via a single command: `docker compose up -d`.
- **NFR-MAINT-02 (API Documentation):** Every exposed endpoint is documented and testable via interactive Swagger UI at `/api-docs`.

6. Requirements Traceability Matrix (RTM)

The Requirements Traceability Matrix guarantees that all functional requirements map directly to implementation modules and automated test cases:

Req ID	Requirement Name	Architecture Component	Source Code Implementation	Verification Method & Test Case
FR-AUTH-01	LINE LIFF & Login Auth	Auth Controller	<code>line.routes.js</code> , <code>line.services.js</code>	Integration test via mock ID token
FR-AUTH-02	Google OAuth PKCE SSO	OAuth Helper	<code>google.routes.js</code> , <code>oauth.helpers.js</code>	<code>tests/oauth.helpers.test.js</code>
FR-AUTH-03	Staff Admin Login	Auth Middleware	<code>staffAuth.routes.js</code> , <code>verifyAdminJwt.js</code>	Unit test: Bcrypt hash comparison
FR-BOOK-01	Interactive Booking Wizard	Client Frontend	<code>src/app/booking/page.tsx</code>	E2E test: Reservation wizard flow
FR-BOOK-02	Dynamic Duration Pricing	Validation Service	<code>bookingCatalog.ts</code> , <code>booking.validation.js</code>	<code>tests/booking.sync.test.js</code>
FR-BOOK-03	Voucher Discount Engine	Voucher Service	<code>Vouchers.js</code> , <code>userVoucher.routes.js</code>	Unit test: Discount arithmetic
FR-BOOK-04	Email Notifications	Mail Utility	<code>sendNotifications.js</code> , <code>nodemailer</code>	<code>tests/notification.recipient.test.js</code>
FR-BOOK-05	Customer Cancellation	Booking Service	<code>userbooking.services.js</code> , <code>profile/page.tsx</code>	Unit test: Status transition to cancelled
FR-BRAN-01	Geolocation Haversine	Client Section	<code>BranchesSection.tsx</code> , <code>Branch.js</code>	Mathematical proximity calculation test
FR-FBOT-01	Meta Messenger Webhook	Webhook Service	<code>facebook.services.js</code> , <code>check-facebook-bot.ps1</code>	Automated script: <code>check-facebook-bot.ps1</code>
FR-ADMN-01	Master Booking Calendar	Calendar Service	<code>calendar.routes.js</code> , <code>bookingCalendar/</code>	Integration test: Monthly density query
FR-ADMN-02	Manual Booking Entry	Admin Controller	<code>manualBooking/page.tsx</code> , <code>booking.routes.js</code>	Integration test: Front-desk walk-in save
FR-ADMN-03	Master Data CRUD	Admin Controllers	<code>branch.routes.js</code> , <code>package.routes.js</code>	Integration test: Package and branch CRUD
FR-ADMN-04	Review Moderation	Review Service	<code>review.routes.js</code> , <code>reviewManagement/</code>	<code>tests/review.sync.test.js</code>
FR-I18N-01	Quadrilingual i18n	Typography Hook	<code>LocaleFont.tsx</code> , <code>public/locales/**</code>	Manual & dictionary completeness test

7. Requirement Verification & Acceptance Sign-Off

This Software Requirements Specification has been inspected and verified against the ISO/IEC/IEEE 29148 requirements engineering standard:

Evaluation Role	Full Name	Signature	Verification Date
Chief Solutions Architect	Mawin Skalet	_____	___ / ___ / 2026
Lead QA & Testing Engineer	Quality Assurance Lead	_____	___ / ___ / 2026
Product Engineering Lead	Product Manager	_____	___ / ___ / 2026